

PEKEGNO

MANAGEMENT SYSTEM

Playbook d'exécution

Spécifications Backend & Frontend par sprint et par tâche

Destinataires	Agents de développement (opencode ou humain)
Usage	« Exécute le sprint S3 » ou « Exécute la tâche T2.1 »
Référentiel	Cahier des charges fonctionnel (Août 2026)
Équipe	Blackstar & Le_million (alternance BE/FE)

Table des matières

1	Objet et mode d'emploi du document	4
1.1	Grammaire de commandes	4
1.2	Règles d'or (à lire avant toute exécution)	4
2	Environnement et commandes	4
2.1	Backend (Laravel 13, PHP 8.3, PostgreSQL)	4
2.2	Frontend (React 19, Vite 6, Tailwind v4)	5
2.3	Comptes de test	5
3	Conventions obligatoires	5
3.1	Base de données	5
3.2	API backend	6
3.3	Frontend	6
4	Architecture cible : navigation et contexte	7
4.1	Modèle organisationnel corrigé	7
4.2	Barre de contexte permanente (ContextBar)	7
4.3	Sidebars dynamiques par type de département	8
4.4	Correspondance des routes (ancien → nouveau)	8
5	Modèle de données cible	8
5.1	M0 — Organisation (Sprint 1)	8
5.2	M1 — Trésorerie (Sprint 2)	8
5.3	M2 — Dépenses (Sprint 3)	9
5.4	M3 — Commissions versionnées (Sprint 3)	9
5.5	M4 — CRM complet (Sprint 4)	10
5.6	M5 — Academy compléments (Sprint 5)	11
5.7	M6 — Contrats Agency & renouvellements (Sprint 5)	11
5.8	M7 — Store / Matootoo (Sprints 6–7)	11
5.9	M8 — Studio (Sprints 8–9)	13
5.10	M9 — Notifications centralisées (Sprint 9)	14
5.11	M10 — KPI, exports, outillage migration (Sprint 10)	14
6	Endpoints API cibles	15
7	Planning et alternance	17
8	Protocole d'exécution d'une tâche	17
8.1	Checklist Definition of Done globale	18
9	SPRINT 1 — Recadrage organisationnel & navigation contextuelle	18
9.1	Tâche T0.1 — Départements typés (BE, BS, 3 j)	18
9.2	Tâche T0.2 — Arbre de contexte enrichi (BE, BS, 3 j)	18

9.3	Tâche T0.6 — Seeders & Swagger (BE, BS, 2 j)	18
9.4	Tâche T0.3 — OrgContext + ContextBar (FE, LM, 5 j)	19
9.5	Tâche T0.4 — Suppression du switcher de lignes + landing Départements (FE, LM, 4 j)	19
9.6	Tâche T0.5 — Départements comme espaces de travail (FE, LM, 4 j + QA partagé)	19
10	SPRINT 2 — Trésorerie & bilan journalier	19
10.1	Tâche T1.1 — Comptes & mouvements de trésorerie (BE, LM, 4 j)	19
10.2	Tâche T1.2 — Bilan journalier généré (BE, LM, 3 j)	20
10.3	Tâche T1.4 — Encaissement avec compte (FE, BS, 3 j)	20
10.4	Tâche T1.5 — Page Trésorerie (FE, BS, 5 j)	20
10.5	Tâche QA-C1 — QA binôme (4 j)	20
11	SPRINT 3 — Dépenses & moteur de commissions	20
11.1	Tâche T2.1 — Module dépenses (BE, BS, 4 j)	20
11.2	Tâche T2.2 — Règles de commissions versionnées (BE, BS, 4 j)	20
11.3	Tâche T2.3 — UI Dépenses (FE, LM, 5 j)	21
11.4	Tâche T2.4 — UI Commissions (FE, LM, 4 j)	21
11.5	Tâche QA-C2/C3 — QA binôme (1 j)	21
12	SPRINT 4 — CRM complet	21
12.1	Tâche T4.1 — Opportunités & pipeline (BE, LM, 3 j)	21
12.2	Tâche T4.2 — Activités & relances (BE, LM, 3 j)	21
12.3	Tâche T4.3 — Entreprises & dédoublonnage (BE, LM, 2 j)	21
12.4	Tâche T4.4 — Kanban prospects (FE, BS, 4 j)	21
12.5	Tâche T4.5 — Fiche prospect complète (FE, BS, 3 j)	22
12.6	Tâche T4.6 — Timeline client (FE, BS, 2 j)	22
12.7	Tâche QA-C4 — QA binôme (1 j)	22
13	SPRINT 5 — Academy compléments & Contrats Agency	22
13.1	Tâche T5.1 — Présences par session (BE, BS, 2 j)	22
13.2	Tâche T5.2 — Certificats (BE, BS, 2 j)	22
13.3	Tâche T6.1 — Contrats & renouvellements (BE, BS, 4 j)	22
13.4	Tâche T5.3 — UI Academy compléments (FE, LM, 5 j)	22
13.5	Tâche T6.2 — UI Contrats & Renouvellements (FE, LM, 4 j)	22
13.6	Tâche QA-C5/C6 — QA binôme (1 j)	23
14	SPRINTS 6–7 — Store / Matootoo	23
14.1	Tâche T7.1 — Fournisseurs & achats (BE, LM, 4 j)	23
14.2	Tâche T7.2 — Stocks (BE, LM, 4 j)	23
14.3	Tâche T7.3 — UI Catalogue & Stocks (FE, BS, 5 j)	23
14.4	Tâche T7.4 — UI Fournisseurs & Achats (FE, BS, 4 j)	23
14.5	Tâche T7.5 — Commandes, livraisons, retours (BE, BS, 4 j)	23

14.6	Tâche T7.6 — Inventaires (BE, BS, 3 j)	23
14.7	Tâche T7.7 — Rapports store (BE, BS, 1 j)	23
14.8	Tâche T7.8 — Ventes Store POS (FE, LM, 3 j)	24
14.9	Tâche T7.9 — UI Commandes/Livraisons/Retours (FE, LM, 4 j)	24
14.10	Tâche T7.10 — UI Inventaires & rapports (FE, LM, 2 j)	24
14.11	Tâche QA-C7 — QA binôme (2+1 j)	24
15	SPRINTS 8–9 — Studio & Notifications	24
15.1	Tâche T8.1 — Devis (BE, LM, 4 j)	24
15.2	Tâche T8.2 — Projets, briefs, planning (BE, LM, 4 j)	24
15.3	Tâche T8.3 — UI Devis & Projets (FE, BS, 5 j)	24
15.4	Tâche T8.4 — UI Briefs & Planning (FE, BS, 4 j)	24
15.5	Tâche T8.5 — Production, révisions, livraisons, clôture (BE, BS, 4 j)	25
15.6	Tâche T8.6 — UI Production/Révisions/Livraisons (FE, LM, 5 j)	25
15.7	Tâche T9.1 — Centre de notifications (BE, BS, 4 j)	25
15.8	Tâche T9.2 — Cloche & centre (FE, LM, 4 j)	25
15.9	Tâche QA-C8/C9 — QA binôme (1+1 j)	25
16	SPRINT 10 — Command Center, rapports & migration	25
16.1	Tâche T10.1 — API KPI consolidée (BE, LM, 4 j)	25
16.2	Tâche T10.2 — Exports & importeur Sheets (BE, LM, 4 j)	25
16.3	Tâche T10.3 — Command Center (FE, BS, 5 j)	25
16.4	Tâche T10.4 — Rapports transverses (FE, BS, 3 j)	26
16.5	Tâche T10.5 — Recette générale (QA, binôme, 2 j)	26
17	Annexe — Interdits et pièges connus	26

1 Objet et mode d'emploi du document

Ce document est la **source de vérité technique** pour le développement restant de l'application PEKEGNO Management System. Il découpe le travail en **sprints** (S1 à S10) contenant des **tâches** identifiées (T0.1, T1.2, ...), chacune spécifiée avec ses modifications de base de données, ses endpoints, ses fichiers à créer ou modifier, et ses critères d'acceptation vérifiables.

1.1 Grammaire de commandes

Un agent de développement (ou un collègue utilisant opencode) peut donner l'une des instructions suivantes :

Instruction	Comportement attendu de l'agent
Exécute le sprint S3	Exécuter séquentiellement toutes les tâches du sprint 3, dans l'ordre de leur numéro, en respectant intégralement le protocole de la section 8.
Exécute la tâche T2.1	Exécuter uniquement cette tâche, même protocole.
Reprend la tâche T7.5	Vérifier d'abord l'état du dépôt (ce qui existe déjà), compléter ce qui manque, puis finir la tâche.
Vérifie le sprint S2	Passer en revue tous les critères d'acceptation du sprint et produire un rapport de conformité sans rien modifier.

1.2 Règles d'or (à lire avant toute exécution)

1. **Une tâche à la fois.** Ne jamais commencer une tâche tant que la précédente du même sprint n'est pas terminée et validée.
2. **Ce document prime.** En cas de conflit entre le code existant et ce document, ce document gagne ; signaler l'écart dans le rapport final.
3. **Zéro destruction.** Jamais de migration destructive sur des tables en production (toujours additive : ajout de colonnes, de tables). Renommer interdit ; déprécier autorisé.
4. **Rien codé en dur.** Pays, agences, départements, catégories, seuils et statuts viennent de la base ou de settings.
5. **Vente $\neq$ paiement.** Une vente possède N paiements ; interdiction absolue de créer des colonnes type « tranche1 / tranche2 ».
6. **Périmètre de tâche strict.** Ne pas retoucher des modules hors du périmètre de la tâche en cours, sauf point d'intégration explicitement décrit.

2 Environnement et commandes

2.1 Backend (Laravel 13, PHP 8.3, PostgreSQL)

Le backend tourne via l'image Docker locale `pekegno-php`. Depuis la racine du projet :

```
# Installer les dependances PHP
sudo docker run --rm -v "$PWD":/app -w /app --network=host pekegno-php composer install --no-interaction

# Reinitialiser complètement la base (migrations + seeders)
cd backend && sudo docker run --rm -v "$PWD":/app -w /app --network=host pekegno-php php artisan migrate:fresh --seed --force

# Regenerer la documentation Swagger apres toute modification d'API
sudo docker run --rm -v "$PWD":/app -w /app --network=host pekegno-php php artisan l5-swagger:generate --all

# Lancer les tests
sudo docker run --rm -v "$PWD":/app -w /app --network=host pekegno-php php artisan test
```

```
# Lister les routes / shell interactif
sudo docker run --rm -v "$PWD":/app -w /app --network=host pekegno-php php artisan route:list
sudo docker run --rm -it -v "$PWD":/app -w /app --network=host pekegno-php php artisan tinker

# Inspecter les tables PostgreSQL
sudo -u postgres psql -d pekegno -c "\dt"
sudo -u postgres psql -d pekegno -c "\d treasury_accounts"
```

2.2 Frontend (React 19, Vite 6, Tailwind v4)

```
cd frontend
npm install          # premiere fois uniquement
npm run dev          # serveur de dev, proxy /api -> http://localhost:8000
npm run build        # DOIT passer sans erreur TypeScript (validation obligatoire)
```

Le build frontend (`npm run build`) est le validateur minimal de chaque tâche frontend : il compile TypeScript en mode strict.

2.3 Comptes de test

Après `migrate:fresh --seed` : connexion staff `admin@pekegno.com` / `password` (rôle super-admin, accès total). Dix utilisateurs de démonstration existent avec différents rôles (voir `UserSeeder`).

3 Conventions obligatoires

Ces conventions reflètent le code existant. Les respecter à la lettre garantit l'homogénéité.

3.1 Base de données

- Clé primaire : `$table->uuid('id')->primary()` ; modèles avec trait `HasUuids`.
- Clés étrangères explicites : `$table->foreign('x_id')->references('id')->on('table')->onDelete('restrictOnDelete() / cascadeOnDelete() justifié)`.
- Montants : `decimal('montant', 14, 2)`. Devise implicite FCFA, jamais stockée par ligne.
- Statuts métier : `string` + contrainte `CHECK PostgreSQL` + constantes publiques dans le modèle (`const STATUS_DRAFT = 'draft'`). Jamais de texte libre.
- Entités métier : `timestamps()` + `softDeletes()` + trio d'endpoints `GET /x/trash`, `POST /x/{id}/restore`, `DELETE /x/{id}/force-delete`.
- Index sur chaque FK et sur les colonnes de filtrage fréquent.
- Transactions financières : suppression dure interdite. Annulation traçable (`cancelled_at` + `cancelled_by` + écriture miroir si trésorerie impactée).
- Commentaires français dans les migrations (`->comment('...')`) pour toute ambiguïté.

Gabarit de migration canonique (respecter ce style) :

```
<?php

use Illuminate\Database\Migrations\Migration;
use Illuminate\Database\Schema\Blueprint;
use Illuminate\Support\Facades\Schema;

return new class extends Migration
{
    public function up(): void
    {
        Schema::create('exemple_tables', function (Blueprint $table) {
            $table->uuid('id')->primary();
            $table->string('name', 150);
            $table->uuid('agency_id');
            $table->foreign('agency_id')->references('id')->on('agencies')->cascadeOnDelete();
            $table->string('status', 20)->default('draft');
```

```

        $table->timestamps();
        $table->softDeletes();

        $table->index('agency_id');
        $table->index('status');
    });

    // Contrainte CHECK spécifique PostgreSQL si nécessaire
    DB::statement("ALTER TABLE exemple_tables ADD CONSTRAINT exemple_tables_status_check
        CHECK (status IN ('draft','active','closed'))");
}

public function down(): void
{
    Schema::dropIfExists('exemple_tables');
}
};

```

3.2 API backend

- Toutes les routes staff vivent dans le groupe middleware ['auth:sanctum', 'single.session', 'inactivit de routes/api.php].
- Contrôle d'accès par middleware `permission:<entite>.<action>`. Actions normalisées (9, voir `PermissionSeeder`) : `créer`, `modifier`, `supprimer`, `consulter`, `exporter`, `imprimer`, `valider`, `encaisser`, `annuler`. Toute nouvelle entité doit déclarer ses permissions dans `PermissionSeeder` et les mapper aux rôles dans `RoleSeeder`.
- Périmètres de visibilité via `App\Services\ScopeService` : un utilisateur non-global ne voit que ses agences/départements assignés. Appliquer systématiquement dans les `index` et filtres.
- Validation inline `$request->validate([...])` avec messages français ; erreurs 422 au format Laravel.
- Logique réutilisable dans `app/Services/*` (ex. `PaymentService`, `CommissionService`) ; contrôleurs minces.
- Journalisation des actions sensibles : trait `App\Observers\Concerns\LogsActivity` ou `ActivityLogger` (paiements, annulations, validations, changements de permissions).
- Annotations Swagger `@OA\*` obligatoires sur chaque endpoint public, puis régénérer (`15-swagger:generate`).
- Listes paginées : `per_page` (défaut 15), réponse au format `paginate` Laravel.

3.3 Frontend

- Un module API par domaine dans `src/api/<domaine>.api.ts`, typé via `src/types/<domaine>.ts`, consommant client de `src/api/client.ts`. Gabarit :

```

import { client } from './client';
import type { Expense, ExpenseListParams } from '@types/expense';
import type { PaginatedResponse } from '@types/agency';

export const expensesApi = {
    async list(params: ExpenseListParams = {}): Promise<PaginatedResponse<Expense>> {
        const { data } = await client.get<PaginatedResponse<Expense>>('/expenses', { params });
        return data;
    },
    async create(payload: Partial<Expense>): Promise<Expense> {
        const { data } = await client.post<Expense>('/expenses', payload);
        return data;
    },
};

```

- Pages dans `src/pages/<module>/`, importées en *lazy* dans `src/router/index.tsx` avec fallback `SkeletonPage` (suivre le motif existant).
- Composants UI exclusivement depuis `src/components/ui/` (`Button`, `Input`, `Select`, `Modal`, `ConfirmDialog`, `Badge`, `Pagination`, `PeriodPicker`, `Autocomplete`, `Alert`, `Skeleton...`). Pas de nouvelle dépendance UI sans décision.

- Menus latéraux : `src/components/layout/navItems.ts` (global) et fonctions par périmètre (`getAgencyItems`, `getDepartmentItems`...). Chaque entrée : `{ to, label: t('nav.cle'), icon: IconName }`
- `i18n` : les deux fichiers `src/i18n/locales/fr.ts` et `en.ts` doivent contenir chaque nouvelle clé. Aucun texte littéral dans les composants.
- Montants : `formatCurrency` (`src/utils/number.ts`); dates : utilitaires de `src/utils/date.ts`.
- Permissions côté client : utilitaires purs dans `src/utils/*Permissions.ts` calqués sur les politiques backend (indicatifs, le backend reste maître).

4 Architecture cible : navigation et contexte

4.1 Modèle organisationnel corrigé

```
PEKEGNO GROUP (organization)
|-- country (ex: Cameroun)
|   |-- agency (ex: Douala)           <- structure physique
|       |-- department type=academy
|       |-- department type=agency    <- branche prestations
|       |-- department type=store
|       |-- department type=studio
```

Entrer dans une agence amène sur la page **Départements**. On y crée un département de l'un des quatre types. La notion actuelle d'« agence academy/mixed » (`agencies.type`) est **dépréciée** : ne plus ni l'écrire ni la lire. Le switcher « Ligne Prestations / Ligne Formations » de `AgencyLayout.tsx` est supprimé.

4.2 Barre de contexte permanente (ContextBar)

Présente en haut de toutes les vues authentifiées :

```
PEKEGNO | Pays ▼ | Agence ▼ | Département ▼ | Période ▼ | Recherche
          | Notifications | Profil
```

- Alimentée par un arbre chargé une fois via `GET /scope/context` (forme ci-dessous), mis en cache dans un provider React global `OrgContext` (`src/context/OrgContext.tsx`) persisté en `localStorage`.
- Sélection à n'importe quel niveau, y compris saut direct vers un autre pays : choisir « Côte d'Ivoire → Abidjan » depuis Cameroun/Yaoundé/Academy doit rediriger proprement vers le nouveau contexte.
- La sélection pilote l'URL (routes paramétrées) et la période pilote tous les KPI.

```
// GET /scope/context : forme cible
{
  "user": {"is_global": true, "assignments": [...]},
  "countries": [
    {
      "id": "...", "name": "Cameroun", "code": "CMR",
      "agencies": [
        {
          "id": "...", "name": "Yaounde", "code": "AG002",
          "departments": [
            {"id": "...", "name": "Academy Yaounde", "type": "academy"},
            {"id": "...", "name": "Store Yaounde", "type": "store"}
          ]
        }
      ]
    }
  ]
}
```


4.3 Sidebars dynamiques par type de département

`getDepartmentItems(type)` retourne exactement ces menus :

- **academy** : Dashboard · Prospects · Apprenants · Inscriptions · Formations (Catalogue, Sessions, Planning, Formateurs) · Présences · Paiements · Créances · Certificats · Performances commerciales · Rapports
- **agency** : Dashboard · Prospects · Clients · Packages · Contrats · Prestations · Équipe client · Community Management · Publicité · Renouvellements · Paiements · Créances · Rapports clients · Performance
- **store** : Dashboard · Catalogue / Produits / Catégories · Stocks · Approvisionnements · Fournisseurs · Achats · Commandes · Ventes · Livraisons · Retours · Inventaires · Rapports
- **studio** : Dashboard · Prospects / Clients · Services · Devis · Projets · Briefs · Planning · Production · Révisions · Livraisons · Paiements · Rapports

Les pages existantes d'Academy (7 pages sous `src/pages/academy/`) sont **réutilisées telles quelles** mais re-racines sous `/departments/:departmentId/...`

4.4 Correspondance des routes (ancien → nouveau)

Ancienne route	Nouvelle route
<code>/countries/:countryId/agencies/:agencyId</code> (overview lignes)	<code>/countries/:countryId/agencies/:agencyId</code> (page Départements = landing)
<code>.../agencies/:agencyId/academy/*</code>	<code>/departments/:departmentId/(dashboard prospects learners courses sessions trainers</code>
<code>.../agencies/:agencyId/services</code>	<code>/departments/:departmentId/services</code> (dept type agency)
<code>/departments/:departmentId</code> (actuel, 3 menus)	<code>/departments/:departmentId/dashboard</code> + menus dynamiques selon type

Redirections permanentes des anciennes URLs maintenues pendant toute la durée du projet (composant de redirection dédié).

5 Modèle de données cible

Spécification DDL complète des évolutions. Types abrégés : `uuid` = UUID PK/FK, `str(n)` = `varchar(n)`, `dec` = `decimal(14,2)`, `ts` = `timestamps`, `sd` = `softDeletes`.

5.1 M0 — Organisation (Sprint 1)

```
ALTER departments
+ type          str(20) NOT NULL DEFAULT 'agency'
                CHECK IN ('academy','agency','store','studio'), INDEX
Backfill : departments heritant de agences.type
          ('academy' -> academy, sinon 'agency').
DEPRECATED : agences.type (ne plus lire/crire).

ALTER invoices  + department_id uuid NULL -> departments.id
ALTER orders    + department_id uuid NULL -> departments.id
                (contexte "branche" de chaque vente, requis par le noyau de ventes CDC)
```

5.2 M1 — Trésorerie (Sprint 2)

```
treasury_accounts
id uuid PK
```

```

agency_id      uuid NULL -> agencies.id nullOnDelete (NULL = compte groupe)
name           str(100)
type           str(20) CHECK IN ('cash','mobile_money','bank')
provider       str(50) NULL -- orange_money, mtn_momo, afriland, cca, other
account_number str(50) NULL
opening_balance dec DEFAULT 0
currency_code  str(3) DEFAULT 'XAF'
is_active      bool DEFAULT true
ts + sd ; UNIQUE(agency_id, name) ; INDEX(type)

treasury_transactions
id uuid PK
treasury_account_id uuid -> treasury_accounts.id restrictOnDelete
direction      str(10) CHECK IN ('in','out')
amount         dec > 0
source_type    str(50) NULL -- morph: invoice_payment, expense, transfer, manual
source_id      uuid NULL
category       str(50) NULL -- vente, approvisionnement, salaire, autre
label          str(200)
reference      str(100) NULL
transacted_at  timestamp
created_by     uuid -> users.id nullOnDelete
ts ; INDEX(account_id, transacted_at) ; INDEX(source_type, source_id)

ALTER invoice_payments + treasury_account_id uuid NULL -> treasury_accounts.id NULL
(backfill : compte cash de l'agence de la facture)

```

Invariant : le solde d'un compte n'est **jamais stocké** ; il vaut $\text{opening_balance} + \text{SUM(in)} - \text{SUM(out)}$.

5.3 M2 — Dépenses (Sprint 3)

```

expenses
id uuid PK
number      str(30) UNIQUE -- EXP-00001
agency_id   uuid -> agencies.id cascadeOnDelete
department_id uuid NULL -> departments.id nullOnDelete
category_id uuid -> accounting_categories.id restrictOnDelete
            (categories type='expense' uniquement, valide au save)
amount      dec > 0
expense_date date
status      str(20) DEFAULT 'draft'
            CHECK IN ('draft','submitted','approved','rejected','paid','closed')
requested_by uuid -> users.id (demandeur)
approved_by  uuid NULL -> users.id ; approved_at ts NULL
rejected_by  uuid NULL -> users.id ; rejection_reason str(255) NULL
paid_by      uuid NULL -> users.id ; paid_at ts NULL
treasury_account_id uuid NULL -> treasury_accounts.id (compte debite au paiement)
justification_path str(255) NULL -- upload (photo/PDF)
note        text NULL
cancelled_at  ts NULL ; cancelled_by uuid NULL -> users.id
ts + sd ; INDEX(status), INDEX(agency_id, expense_date)

Workflow : draft -> submitted -> approved -> paid -> closed
          \-> rejected (raison obligatoire)
Transitions controlees dans ExpenseService ; passage a 'paid' =
écriture treasury_transactions(direction=out) + accounting_transactions
(categorie depense) automatiques, atomique (DB::transaction).

```

5.4 M3 — Commissions versionnées (Sprint 3)

```

commission_rules
id uuid PK
rule_group_id uuid -- identite de la regle a travers ses versions
version      int DEFAULT 1 -- incremente a chaque edition
name         str(150)
beneficiary_commercial_id uuid NULL -> commercials.id nullOnDelete
            -- beneficiaire designe OU portee generique ci-dessous
scope_country_id  uuid NULL -> countries.id
scope_agency_id   uuid NULL -> agencies.id
scope_department_id uuid NULL -> departments.id

```

```

service_id          uuid NULL -> services.id -- NULL = tout produit/service
trigger_event       str(20) CHECK IN ('on_sale','on_payment','on_full_payment')
formula_type        str(10) CHECK IN ('percent','fixed','tiered')
percent_value       dec(5,2) NULL -- si percent
fixed_amount        dec NULL -- si fixed
tiers_json          jsonb NULL -- si tiered : [{up_to, value, mode}]
starts_on / ends_on date NULL
is_active           bool DEFAULT true
created_by          uuid -> users.id
ts + sd ; INDEX(rule_group_id, version) UNIQUE(rule_group_id, version)

```

IMMUTABILITE : une regle publiee n'est jamais modifiee en place ;
l'edition cree une nouvelle ligne (version+1, meme rule_group_id)
et desactive la precedente.

```

commission_entries
id uuid PK
invoice_id          uuid -> invoices.id cascadeOnDelete
invoice_payment_id uuid NULL -> invoice_payments.id nullOnDelete
commission_rule_id uuid -> commission_rules.id restrictOnDelete
rule_snapshot       jsonb -- copie integrale de la regle appliquee
beneficiary_commercial_id uuid -> commercials.id
base_amount         dec
amount              dec
status              str(20) DEFAULT 'calculated'
                   CHECK IN ('calculated','validated','paid','cancelled')
validated_by uuid NULL ; validated_at ts NULL
paid_by            uuid NULL ; paid_at      ts NULL
ts ; INDEX(invoice_id), INDEX(beneficiary..., status)

```

Plusieurs lignes possibles sur une meme vente/paiement.
CommissionService : a l'encaissement, evaluer TOUTES les regles actives
matchant le contexte ; conserver le fallback taux-fiche-commercial
(commission-payments existant) tant que aucune regle ne matche.

5.5 M4 — CRM complet (Sprint 4)

```

companies
id uuid PK ; name str(150) ; industry str(100) NULL
phone/email/address NULL ; city str(100) NULL ; country str(100) NULL
website str(150) NULL ; ts + sd ; INDEX(name)

ALTER prospects + company_id uuid NULL -> companies.id nullOnDelete

opportunities
id uuid PK
title str(200)
prospect_id        uuid NULL -> prospects.id cascadeOnDelete
client_id          uuid NULL -> users.id nullOnDelete
company_id         uuid NULL -> companies.id nullOnDelete
agency_id          uuid -> agencies.id cascadeOnDelete
department_id      uuid NULL -> departments.id nullOnDelete
commercial_id      uuid -> commercials.id cascadeOnDelete
stage              str(20) DEFAULT 'new' CHECK IN
                   ('new','contacted','qualified','proposal','negotiation','won','lost')
expected_amount    dec NULL ; expected_close_date date NULL
won_at / lost_at   ts NULL
ts + sd ; INDEX(stage), INDEX(commercial_id)

activities
id uuid PK
subject_type str(50) ; subject_id uuid -- morph: Prospect, Opportunity, User(client)
assigned_to       uuid -> users.id nullOnDelete
created_by        uuid -> users.id nullOnDelete
type              str(20) CHECK IN ('call','meeting','email','whatsapp','note','followup')
title             str(200) ; notes text NULL
due_at           ts NULL ; completed_at ts NULL
outcome           str(255) NULL
ts ; INDEX(subject_type, subject_id), INDEX(assigned_to, due_at)

Job quotidien (routes/console.php) : rappels des activites echues non
completees -> notifications.

```

Dedoublonnage clients/prospects : recherche par telephone normalise
(chiffres seulement, suffixe 8 derniers chiffres).

5.6 M5 — Academy compléments (Sprint 5)

```
attendances
  id uuid PK
  training_session_id uuid -> training_sessions.id cascadeOnDelete
  enrollment_id        uuid -> enrollments.id cascadeOnDelete
  status str(10) DEFAULT 'present' CHECK IN ('present','absent','late','excused')
  recorded_by uuid -> users.id nullOnDelete ; recorded_at ts
  ts ; UNIQUE(training_session_id, enrollment_id)

certificates
  id uuid PK
  enrollment_id uuid -> enrollments.id cascadeOnDelete UNIQUE
  number str(30) UNIQUE -- CERT-00001
  issued_on date ; mention str(100) NULL
  status str(10) DEFAULT 'issued' CHECK IN ('issued','revoked')
  revoked_reason str(255) NULL
  file_path str(255) NULL -- PDF genere/imprimable
  created_by uuid -> users.id nullOnDelete
  ts + sd

ALTER enrollments + invoice_id uuid NULL -> invoices.id nullOnDelete
(facture d'inscription ; solde apprenant = calcule depuis cette facture
et ses paiements valides ; JAMAIS ressaisi)
```

5.7 M6 — Contrats Agency & renouvellements (Sprint 5)

```
contracts
  id uuid PK
  number str(30) UNIQUE -- CTR-00001
  client_id    uuid -> users.id restrictOnDelete (role client)
  company_id   uuid NULL -> companies.id nullOnDelete
  agency_id    uuid -> agencies.id cascadeOnDelete
  department_id uuid NULL -> departments.id nullOnDelete (dept type agency)
  pack_id      uuid NULL -> subscription_packs.id nullOnDelete
  start_date / end_date date
  billing_cycle str(20) DEFAULT 'monthly'
                  CHECK IN ('one_shot','monthly','quarterly','yearly')
  amount dec
  status str(20) DEFAULT 'active' CHECK IN
    ('active','due_soon','expired','suspended','terminated')
  auto_renew bool DEFAULT false ; renewal_count int DEFAULT 0
  parent_contract_id uuid NULL -> contracts.id nullOnDelete (chaine historique)
  notes text NULL ; terminated_at ts NULL ; terminated_reason str(255) NULL
  ts + sd ; INDEX(client_id), INDEX(status, end_date)

contract_services
  contract_id uuid -> contracts.id cascadeOnDelete
  service_id  uuid -> services.id restrictOnDelete
  price dec NULL -- surcharge du prix catalogue
  UNIQUE(contract_id, service_id)

ALTER accounting_categories + is_pass_through bool DEFAULT false
-- true = budget publicitaire gere POUR le client : mouvement trace mais
-- EXCLU du chiffre d'affaires (regle comptable CDC section 8.2)

Job quotidien 06:30 : end_date <= J+30 et status='active' -> 'due_soon'
+ notification ; end_date passee -> 'expired' + alerte.
Settings : renew_alert_days jsonb DEFAULT '[30,7]'
```

5.8 M7 — Store / Matootoo (Sprints 6–7)

```
suppliers
  id uuid PK ; code str(20) UNIQUE -- FRN-00001
```

```

name str(150) ; phone/email/address NULL ; city str(100) NULL
notes text NULL ; agency_id uuid NULL -> agencies.id nullOnDelete
is_active bool DEFAULT true ; ts + sd ; INDEX(name)

purchases -- Achats / Approvisionnements
id uuid PK ; number str(30) UNIQUE -- ACH-00001
supplier_id uuid -> suppliers.id restrictOnDelete
agency_id uuid -> agencies.id cascadeOnDelete
department_id uuid NULL -> departments.id nullOnDelete
status str(20) DEFAULT 'draft' CHECK IN
('draft','ordered','partially_received','received','cancelled')
order_date date ; expected_at date NULL ; received_at ts NULL
subtotal dec ; discount dec DEFAULT 0 ; tax_rate dec(5,2) DEFAULT 0
total_amount dec
payment_status str(20) DEFAULT 'unpaid'
CHECK IN ('unpaid','partial','paid')
treasury_account_id uuid NULL -> treasury_accounts.id -- sortie fournisseur
notes text NULL ; created_by uuid -> users.id
ts + sd ; INDEX(supplier_id), INDEX(status, order_date)

purchase_lines
id uuid PK ; purchase_id uuid -> purchases.id cascadeOnDelete
product_id uuid -> products.id restrictOnDelete
qty_ordered dec(12,2) ; qty_received dec(12,2) DEFAULT 0
unit_cost dec ; line_total dec
ts

stock_levels
id uuid PK
product_id uuid -> products.id cascadeOnDelete
agency_id uuid -> agencies.id cascadeOnDelete
department_id uuid NULL -> departments.id nullOnDelete
warehouse str(100) DEFAULT 'principal'
qty_on_hand dec(12,2) DEFAULT 0
reorder_point dec(12,2) DEFAULT 0
ts ; UNIQUE(product_id, agency_id, department_id, warehouse)

stock_movements
id uuid PK
product_id uuid -> products.id restrictOnDelete
agency_id uuid -> agencies.id restrictOnDelete
department_id uuid NULL -> departments.id nullOnDelete
type str(20) CHECK IN ('in','sale','out','return','loss','adjustment',
'transfer_in','transfer_out')
quantity dec(12,2) > 0 -- toujours positif ; le sens vient du type
unit_cost dec NULL ; reference str(100) NULL
source_type str(50) NULL ; source_id uuid NULL -- purchase_line, order_line...
transfer_ref uuid NULL -- couple transfert (out/in) partage cet uuid
moved_at ts ; created_by uuid -> users.id
ts ; INDEX(product_id, moved_at), INDEX(type), INDEX(transfer_ref)

StockService : seule porte d'écriture du stock. Dans une DB::transaction :
1) insérer stock_movements 2) recalculer qty_on_hand depuis les mouvements
3) si qty_on_hand < reorder_point -> notification.
Types sortants : sale/out/loss/transfer_out. Entrants : in/return/transfer_in.
adjustment : delta signe via quantity positif + label.

ALTER order_lines + product_id uuid NULL -> products.id nullOnDelete
(vente catalogue store ; line_type='catalog')

deliveries
id uuid PK ; order_id uuid -> orders.id cascadeOnDelete
status str(20) DEFAULT 'pending' CHECK IN
('pending','preparing','shipped','delivered','cancelled')
carrier str(100) NULL ; tracking_number str(100) NULL
delivered_at ts NULL ; received_by str(150) NULL
notes text NULL ; ts ; INDEX(order_id, status)

returns -- Retours produits
id uuid PK ; number str(30) UNIQUE -- RET-00001
order_id uuid NULL -> orders.id nullOnDelete
invoice_id uuid NULL -> invoices.id nullOnDelete
client_id uuid NULL -> users.id nullOnDelete
product_id uuid -> products.id restrictOnDelete
quantity dec(12,2) > 0 ; reason str(20) CHECK IN

```

```

        ('defective','wrong_item','client_remorse')
restock bool DEFAULT true -- reintegre le stock via StockService
refund_amount dec DEFAULT 0 ; refund_method str(20) NULL
treasury_account_id uuid NULL -> treasury_accounts.id -- sortie remboursement
status str(20) DEFAULT 'requested' CHECK IN
    ('requested','accepted','rejected','refunded','closed')
processed_by uuid NULL -> users.id ; processed_at ts NULL
ts + sd ; INDEX(status)

inventories
id uuid PK ; number str(30) UNIQUE -- INV-00001
agency_id uuid -> agencies.id cascadeOnDelete
department_id uuid NULL -> departments.id nullOnDelete
status str(20) DEFAULT 'open' CHECK IN ('open','counting','closed')
started_by uuid -> users.id ; closed_at ts NULL ; note text NULL
ts + sd

inventory_lines
id uuid PK ; inventory_id uuid -> inventories.id cascadeOnDelete
product_id uuid -> products.id restrictOnDelete
expected_qty dec(12,2) -- gele a la creation de la session
counted_qty dec(12,2) NULL
unit_cost dec NULL
adjustment_movement_id uuid NULL -> stock_movements.id nullOnDelete
UNIQUE(inventory_id, product_id)

Cloture inventaire : ecart = counted_qty - expected_qty ; cree un
stock_movement type='adjustment' par ligne en ecart, valorise unit_cost.

```

5.9 M8 — Studio (Sprints 8–9)

```

quotes -- Devis
id uuid PK ; number str(30) UNIQUE -- DEV-00001
client_id uuid NULL -> users.id nullOnDelete
prospect_id uuid NULL -> prospects.id nullOnDelete
company_id uuid NULL -> companies.id nullOnDelete
agency_id uuid -> agencies.id cascadeOnDelete
department_id uuid NULL -> departments.id nullOnDelete
status str(20) DEFAULT 'draft' CHECK IN
    ('draft','sent','accepted','refused','expired')
valid_until date NULL ; sent_at ts NULL ; decided_at ts NULL
subtotal dec ; discount dec DEFAULT 0 ; tax_rate dec(5,2) DEFAULT 0
total_amount dec
converted_project_id uuid NULL -- pose a la conversion (pas de FK directe)
created_by uuid -> users.id
ts + sd ; INDEX(status)

quote_lines
id uuid PK ; quote_id uuid -> quotes.id cascadeOnDelete
label str(200) ; description text NULL
unit_price dec ; quantity dec(12,2) DEFAULT 1 ; line_total dec
position int DEFAULT 0 ; ts

projects
id uuid PK ; number str(30) UNIQUE -- PRJ-00001
title str(200) ; description text NULL
client_id uuid -> users.id restrictOnDelete
company_id uuid NULL -> companies.id nullOnDelete
quote_id uuid NULL -> quotes.id nullOnDelete
agency_id uuid -> agencies.id cascadeOnDelete
department_id uuid NULL -> departments.id nullOnDelete
manager_id uuid NULL -> users.id nullOnDelete
status str(20) DEFAULT 'planning' CHECK IN
    ('planning','in_production','in_revision','delivered','closed','cancelled')
start_date date NULL ; due_date date NULL
delivered_at ts NULL ; closed_at ts NULL
budget_amount dec NULL
invoice_deposit_id uuid NULL -> invoices.id nullOnDelete -- acompte
invoice_balance_id uuid NULL -> invoices.id nullOnDelete -- solde
ts + sd ; INDEX(status), INDEX(department_id)

project_briefs -- reunions de cadrage, notes riches

```

```

id uuid PK ; project_id uuid -> projects.id cascadeOnDelete
title str(200) ; meeting_date date
participants text NULL          -- noms separes par virgule (v1)
content_html longtext NULL     -- rendu Summernote (HTML assaini)
created_by uuid -> users.id
ts ; INDEX(project_id)

project_tasks                  -- planning / production
id uuid PK ; project_id uuid -> projects.id cascadeOnDelete
title str(200) ; description text NULL
assignee_id uuid NULL -> users.id nullOnDelete
start_date / due_date date NULL
status str(20) DEFAULT 'todo' CHECK IN ('todo','in_progress','done','blocked')
position int DEFAULT 0 ; completed_at ts NULL
ts ; INDEX(project_id, status)

project_revisions
id uuid PK ; project_id uuid -> projects.id cascadeOnDelete
round int DEFAULT 1
request_note text ; requested_by uuid -> users.id
requested_at ts ; resolved_note text NULL ; resolved_at ts NULL
status str(20) DEFAULT 'pending' CHECK IN ('pending','resolved')
ts ; INDEX(project_id, status)

deliverables
id uuid PK ; project_id uuid -> projects.id cascadeOnDelete
title str(200) ; file_path str(255)
revision_id uuid NULL -> project_revisions.id nullOnDelete
version int DEFAULT 1
uploaded_by uuid -> users.id
delivered_at ts NULL
ts ; INDEX(project_id)

Cycle CDC : Prospect -> Brief -> Devis -> Validation -> Acompte ->
Planification -> Production -> Revision -> Validation client ->
Livraison -> Solde -> Cloture.

```

5.10 M9 — Notifications centralisées (Sprint 9)

```

notifications (table generale, schema natif Laravel ok)
id uuid PK ; user_id uuid -> users.id cascadeOnDelete
type str(50)  -- receivable_overdue, renewal_due, stock_low,
              -- expense_submitted, expense_decided, activity_reminder
title str(200) ; body text NULL
entity_type str(50) NULL ; entity_id uuid NULL
read_at ts NULL
timestamps ; INDEX(user_id, read_at)

NotificationService::notify(users[], type, title, body, entity)
Declencheurs a cabler :
- immediat : expense submitted -> validateurs du perimetre ;
  stock sous seuil (apres StockService) ;
- job quotidien 07:00 : creances echues (factures impayees echues),
  renouvellements contrats/abonnements J-X, rappels activities.

```

5.11 M10 — KPI, exports, outillage migration (Sprint 10)

```

import_logs
id uuid PK ; filename str(255) ; entity str(50)
mapping_version str(50) ; rows_total int ; rows_imported int
rows_skipped int ; errors jsonb ; dry_run bool DEFAULT true
executed_by uuid -> users.id ; ts

Commande : php artisan pekegno:import-sheets {file} {entity} [--dry-run]
Mappings : config/storage json versionne (storage/app/import-maps/*.json)
Regles : jamais d'écriture comptable certifiée sans --force ;
  anomalies journalisées dans errors, lignes ignorees listees.

```

6 Endpoints API cibles

Tous sous préfixe `/api`, groupe staff authentifié. **CRUD** signifie index/store/show/update/destroy + trash/restore/force-delete quand l'entité a `softDeletes`.

TABLE 1: Endpoints nouveaux ou modifiés par sprint.

Verbe	Route	Permission	Description
<i>S1 — Contexte</i>			
GET	/scope/context	(auth)	Arbre pays→agences→départements du périmètre utilisateur, enrichi des types.
PUT	/scope/context-selection	(auth)	Persiste la dernière sélection de contexte de l'utilisateur (colonne <code>settings</code> JSON côté user ou table settings clé <code>context_selection:{userId}</code>).
<i>S2 — Trésorerie</i>			
CRUD	/treasury/accounts	tresories.*	Comptes de trésorerie (cash/MoMo/-banque).
GET	/treasury/accounts/{account}/transactions	tresories.consulter	Mouvements paginés d'un compte, filtres période/type.
POST	/treasury/transfers	tresories.valider	Transfert entre deux comptes (2 mouvements liés).
GET	/treasury/summary	tresories.consulter	Soldes courants par compte + totaux période.
GET	/bilans/daily	bilans.consulter	Bilan journalier généré (ventes par branche, encaissements par compte, dépenses, solde théorique vs réel). Paramètre <code>date</code> .
<i>S3 — Dépenses & commissions</i>			
CRUD	/expenses	depenses.*	Dépenses.
POST	/expenses/{expense}/submit	depenses.modifier	draft → submitted.
POST	/expenses/{expense}/approve	depenses.valider	submitted → approved.
POST	/expenses/{expense}/reject	depenses.valider	submitted → rejected (raison requise).
POST	/expenses/{expense}/pay	depenses.encaisser	approved → paid : sortie trésorerie + écriture comptable.
CRUD	/commission-rules	commissions.*	Règles versionnées (update = nouvelle version).
GET	/commissions/entries	commissions.consulter	Lignes générées, filtres période/statut/-bénéficiaire.
POST	/commissions/entries/{entry}/calculate	commissions.valider	calculated → validated.
POST	/commissions/entries/{entry}/pay	commissions.valider	validated → paid.
<i>S4 — CRM</i>			
CRUD	/companies	entreprises.*	Entreprises.
CRUD	/opportunities	opportunités.*	Opportunités ; PATCH stage inclus.
CRUD	/activities	activités.*	Activités polymorphes ; POST .../complete.
GET	/crm/timeline{?}subject_type,subject_id	crm.consulter	Timeline relationnelle unifiée.
<i>S5 — Academy & contrats</i>			
PUT	/training-sessions/{session}/attendances	presences.modifier	Sauvegarde bulk feuille de présence.

Verbe	Route	Permission	Description
CRUD	/certificates	certificats.*	Émission/révocation ; GET PDF imprimable.
CRUD	/contracts	contrats.*	Contrats Agency + services liés.
POST	/contracts/{contract}/renew	contrats.creer	Renouvelle : nouveau contrat chaîné, ancien → renewed/expired.
POST	/contracts/{contract}/terminate	contrats.annuler	Résiliation motivée.
<i>S6-S7 — Store</i>			
CRUD	/suppliers	fournisseurs.*	Fournisseurs.
CRUD	/purchases	achats.*	Achats/approvisionnements.
POST	/purchases/{purchase}/receive	achats.valider	Réception (qty par ligne) → mouvements in.
POST	/purchases/{purchase}/pay	achats.encaisser	Paie fournisseur → sortie trésorerie.
GET	/stocks/levels	stocks.consulter	Niveaux, filtres produit/agence/département/seuil.
GET	/stocks/movements	stocks.consulter	Journal des mouvements.
POST	/stocks/movements	stocks.creer	Mouvement manuel (ajustement, perte, transfert).
POST	/orders/{order}/deliveries	livraisons.creer	Crée la livraison d'une commande.
PATCH	/deliveries/{delivery}	livraisons.modifier	Statut livraison.
CRUD	/returns	retours.*	Retours ; accept/refund intégrés au workflow.
CRUD	/inventories	inventaires.*	Sessions d'inventaire + lignes.
POST	/inventories/{inventory}/close	inventaires.valider	Clôture : ajustements de stock générés.
GET	/reports/store	rapports.consulter	Rotation, marge, top produits.
<i>S8-S9 — Studio & notifications</i>			
CRUD	/quotes	devis.*	Devis ; POST .../send, /accept, /refuse.
POST	/quotes/{quote}/convert	devis.valider	Convertit en projet (statut planning).
CRUD	/projects	projets.*	Projets studio.
CRUD	/projects/{project}/briefs	briefs.*	Briefs (notes HTML).
CRUD	/projects/{project}/tasks	projets.*	Tâches/planning ; réordonnancement via position.
CRUD	/projects/{project}/revisions	revisions.*	Cycles de révision.
CRUD	/projects/{project}/deliverables	projets.*	Livrables (upload).
POST	/projects/{project}/close	projets.valider	Clôture (solde facturé exigé).
GET	/notifications	(auth)	Liste personnelle, filtre non-lues.
POST	/notifications/read-all	(auth)	Marque tout lu.
PATCH	/notifications/{notification}	(auth)	Marque lu/non lu.
<i>S10 — Command Center & imports</i>			
GET	/command-center	kpi.consulter	Cartes KPI consolidées + comparaison période précédente. Filtres : pays/agence/département/période.
GET	/kpi/drill-down	kpi.consulter	Descente dimensionnelle jusqu'à la transaction.
POST	/imports/preview	kpi.creer	Analyse CSV vs mapping, rapport d'anomalies (dry-run).
POST	/imports/run	kpi.creer	Import définitif journalisé (-force équivalent).

Permissions nouvelles à déclarer dans `PermissionSeeder` (entités) : `tresories`, `depenses`, `commissions` (si absent), `entreprises`, `opportunités`, `activités`, `présences`, `certificats`, `contrats`,

fournisseurs, achats, stocks, livraisons, retours, inventaires, devis, projets, briefs, revisions, notifications, kpi — avec les 9 actions normalisées quand pertinent. Mapping rôles (RoleSeeder) : direction-generale = tout ; responsable-agence / responsable-departement = tout sur leur périmètre ; comptable = tresories, depenses, commissions, bilans ; caissier = tresories consulter+encaisser, depenses consulter+creer ; commercial = crm/opportunites/activites/devis consulter+creer+modifier.

7 Planning et alternance

Deux développeurs, Blackstar (BS) et Le_million (LM). Sprint = 2 semaines ; les rôles BE/FE permutent à chaque sprint :

Sprint	Semaines	Backend	Frontend	Chantier
S1	S1–S2	BS	LM	Navigation & contexte (T0.x)
S2	S3–S4	LM	BS	Trésorerie & paiements (T1.x)
S3	S5–S6	BS	LM	Dépenses + commissions (T2.x)
S4	S7–S8	LM	BS	CRM complet (T4.x)
S5	S9–S10	BS	LM	Academy + contrats (T5.x)
S6	S11–S12	LM	BS	Store 1/2 (T7.x)
S7	S13–S14	BS	LM	Store 2/2 (T7.x)
S8	S15–S16	LM	BS	Studio 1/2 (T8.x)
S9	S17–S18	BS	LM	Studio 2/2 + notifications (T8/T9.x)
S10	S19–S20	LM	BS	Command Center & imports (T10.x)

Un développeur exécute les tâches de son flux (BE ou FE) ; les tâches QA sont binômes. Quand une instruction porte sur un sprint entier, l'agent exécute les tâches correspondant à son flux puis signale celles du binôme.

8 Protocole d'exécution d'une tâche

À appliquer pour CHAQUE tâche, sans exception :

1. **Lire** la carte de tâche et les sections référencées (modèle §5, endpoints §6, navigation §4).
2. **Vérifier l'état** : `git status` propre, `migrate:fresh --seed` passe, `npm run build` passe. Si une dépendance de la carte n'existe pas encore (tâche d'un autre flux non réalisée), implémenter le minimum syndical côté API (contrats définis ici) et le noter dans le rapport.
3. **Backend** (si tâche BE) : migration(s) → modèle(s) avec constantes de statuts et relations → Service si logique métier → contrôleur + routes + middleware permission → ScopeService sur les listes → LogsActivity sur actions sensibles → annotations Swagger → mise à jour PermissionSeeder/RoleSeeder → seeder de démonstration réaliste.
4. **Validation backend** : `migrate:fresh --seed --force` sans erreur, `php artisan test` vert, `l5-swagger:generate --all`, vérifier les nouveaux endpoints via `route:list`.
5. **Frontend** (si tâche FE) : types TS → module `*.api.ts` → page(s) avec kit UI → routes lazy + Skeleton → entrées de menu (navItems ou getDepartmentItems) → i18n fr et en → permissions utilitaires si besoin → états loading/error/empty.
6. **Validation frontend** : `npm run build` vert (TypeScript strict). Smoke test manuel du parcours critique décrit dans les critères.
7. **Commit** : un commit par tâche, message `feat(scope): TX.Y resume` (scope = backend, frontend, ou fullstack).
8. **Rapport de tâche** (sortie finale obligatoire) : liste des fichiers créés/modifiés, endpoints exposés, migrations créées, écarts éventuels avec cette spécification, points ouverts.

8.1 Checklist Definition of Done globale

- ☐ Migrations additives uniquement ; `migrate:fresh --seed --force` passe.
- ☐ Statuts contrôlés (CHECK + constantes), aucun statut libre.
- ☐ Permissions `entite.action` câblées et mappées aux rôles.
- ☐ Périmètre utilisateur respecté sur toutes les listes (ScopeService).
- ☐ Actions sensibles journalisées (`activity_logs`).
- ☐ Swagger régénéré.
- ☐ i18n FR/EN complète, aucun texte en dur.
- ☐ `npm run build` et `php artisan test` verts.
- ☐ Solde/trésorerie/commission : calculés depuis les transactions sources, jamais ressaisis.
- ☐ Aucune donnée (pays, agence, département, catégorie, seuil) codée en dur.

9 SPRINT 1 — Recadrage organisationnel & navigation contextuelle

Objectif : appliquer le modèle Agence → Départements typés et livrer la ContextBar. Références : §4, modèle M0 (§5).

9.1 Tâche T0.1 — Départements typés (BE, BS, 3 j)

Prérequis : aucun.

Travail backend :

1. Migration additive : colonne `type` sur `departments` (M0) + backfill dans la migration (`UPDATE departments` selon `agencies.type`).
2. Modèle `Department` : constantes `TYPE_ACADEMY='academy'` etc., `TYPES` array, validation dans `DepartmentController in:academy,agency,store,studio`.
3. Déprécier `agencies.type` : retirer toute lecture/écriture (modèle `Agency::deriveType`, `TYPES`, affichages), garder la colonne. Commentaire de dépréciation.
4. `DepartmentController@store/update` : champ `type` requis/validé ; réponse inclut `type`.
5. Seeder : `AgencySeeder` crée 4 départements (un par type) par agence de démonstration.
6. Tests feature : création/refus type invalide, backfill correct.

Critères d'acceptation : ☐ \d `departments` montre la colonne `type` + CHECK.

- ☐ POST /departments sans `type` ⇒ 422 ; avec "`store`" ⇒ 201 avec `type` persisté.
- ☐ Aucune référence active à `agencies.type` hors migration historique (grep).

9.2 Tâche T0.2 — Arbre de contexte enrichi (BE, BS, 3 j)

Travail backend : étendre `ScopeController@__invoke` pour retourner la forme JSON de §4 (pays → agences → départements avec `id,name,type`), filtrée par `ScopeService` ; ajouter option `?types=academy,store`. Eager load performant (`with` imbriqué). Tests : un utilisateur assigné à une seule agence ne voit que son sous-arbre. **Critères** : ☐ Réponse conforme à la forme JSON documentée.

- ☐ Temps de réponse < 300 ms avec données de démo.
- ☐ super-admin voit les 2 pays ; commercial limité.

9.3 Tâche T0.6 — Seeders & Swagger (BE, BS, 2 j)

Données de démonstration cohérentes : 2 pays, 3 villes, 3 agences, chacune avec ses 4 départements typés nommés proprement (« Academy Douala »...), utilisateurs de démo assignés à des départements précis. Swagger régénéré.

9.4 Tâche T0.3 — OrgContext + ContextBar (FE, LM, 5 j)

Travail frontend :

1. Nouveau module `src/api/scope.api.ts` (`getContext`, `saveSelection`).
2. Provider `src/context/OrgContext.tsx` : état `{tree, selection:{countryId,agencyId,departmentId,}` persistance `localStorage` (`pekegno_context`), hooks `useOrgContext()`.
3. Composant `src/components/layout/ContextBar.tsx` : 4 dropdowns en cascade (Pays/Agence/Département/Période via `PeriodPicker`), recherche globale (redirige vers page correspondante), cloche notifications (placeholder badge 0, lien futur), profil (réutilise `UserMenu`).
4. Intégration : monter la barre dans `AppLayout`, `CountryLayout`, `AgencyLayout`, `DepartmentLayout` (remplacer les headers back existants). Sélection = navigation vers l'URL canonique du niveau choisi ; changement de pays réinitialise agence/département.

Critères : ☐ Depuis CMR/Yaoundé/Academy, ouvrir « Pays » et choisir Côte d'Ivoire ⇒ redirection immédiate vers Abidjan (ou liste agences CI) sans repasser par le dashboard.

☐ Le contexte survit au refresh (`localStorage` + URL cohérents).

☐ i18n fr/en complète.

9.5 Tâche T0.4 — Suppression du switcher de lignes + landing Départements (FE, LM, 4 j)

1. `AgencyLayout.tsx` : supprimer `AgencyLine`, `navigateLine`, `getAgencyItems/getAcademyItems` et le segmented control ; la sidebar agence devient : Overview · Départements (défaut) · Équipes · Promotions · Abonnements · Services transverses · Settings.
2. Landing agence = page Départements ; cartes de départements montrant le type (icônes : GraduationCap/Briefcase/ShoppingBag/Video), filtres par type.
3. Modale de création : Select type (4 options traduites) + nom + description.

☐ Plus aucune trace de « Ligne Prestations/Formations ».

☐ Création d'un département de chaque type possible depuis l'UI.

9.6 Tâche T0.5 — Départements comme espaces de travail (FE, LM, 4 j + QA partagé)

1. `getDepartmentItems(type)` dans `navItems.ts` retournant les menus §4 ; items non encore implémentés pointent vers des routes placeholder propres (page « À venir – Sprint X » référençant la tâche).
2. Ré-raciner Academy : routes `/departments/:departmentId/{courses,sessions,trainers,learners,...}` réutilisant les composants existants (props `departmentId` au lieu de `agencyId` ; adapter les appels API qui filtrent déjà par `agency_id` via query param).
3. Redirections : `/countries/:c/agencies/:a/academy/*` → département academy de l'agence (composant `LegacyRedirect`).
4. QA binôme : parcours Groupe → Pays → Agence → Département → cours Academy complet.

10 SPRINT 2 — Trésorerie & bilan journalier

Références : modèle M1, endpoints §6. Décisions requises en amont : liste des comptes réels (sinon seeder générique).

10.1 Tâche T1.1 — Comptes & mouvements de trésorerie (BE, LM, 4 j)

1. Migrations M1 (tables + colonnes ajoutées à `invoices`, `orders`, `invoice_payments`) ; backfill `invoice_payments.treasury_account_id` avec le compte cash de l'agence de la facture.

2. Modèles + relations ; constantes `TreasuryAccount::TYPE_*`, `TreasuryTransaction::DIRECTION_*`.
 3. `TreasuryService` : unique porte d'écriture `recordMovement(account, direction, amount, source, ...)` en transaction ; méthode `balanceFor(account)` ; `transfer(from,to,amount) = 2` mouvements liés.
 4. Branchement `PaymentService` : chaque `invoice_payments` enregistre le mouvement in correspondant (montant, source=morph, référence numéro facture).
 5. Contrôleurs + routes + permissions `tresories.*` ; Swagger ; tests (paiement $\Rightarrow$ solde juste ; annulation facture $\Rightarrow$ contrepassation).
- Encaissement de 3 tranches sur une facture $\Rightarrow$ 3 mouvements in, solde exact.
- Impossible de payer une dépense depuis un compte inexistant/inactif (422).

10.2 Tâche T1.2 — Bilan journalier généré (BE, LM, 3 j)

Refactorer `BilanController` : GET `/bilans/daily?date=&agency_id=` calcule depuis les sources : ventes par département type (via `invoices.department_id`), encaissements par compte type, dépenses payées, solde théorique initial + in - out ; compare à `daily_balances.solde_final` saisi (écart +

10.3 Tâche T1.4 — Encaissement avec compte (FE, BS, 3 j)

Formulaire d'encaissement (`InvoiceDetail` + `QuickSale`) : Select compte de trésorerie (filtré agence, groupé par type avec icônes Banknote/Smartphone/Landmark), champ référence ; affichage colonne compte dans l'historique paiements. Types TS + i18n.

10.4 Tâche T1.5 — Page Trésorerie (FE, BS, 5 j)

Menu FINANCE (global + pays + agence) : cartes soldes par compte, tableau flux paginé avec filtres (compte/type/période/Direction), modale transfert entre comptes, accès bilan jour. Permission `tresories.consulter`.

10.5 Tâche QA-C1 — QA binôme (4 j)

Parcours CDC : vente 100 000 FCFA $\rightarrow$ 3 paiements $\rightarrow$ trésorerie $\rightarrow$ bilan cohérent ; transfert inter-comptes ; écarts affichés.

11 SPRINT 3 — Dépenses & moteur de commissions

Références : modèles M2, M3. Décision requise : formules réelles de prime (sinon règles de démo).

11.1 Tâche T2.1 — Module dépenses (BE, BS, 4 j)

Tables M2 ; `ExpenseService` (transitions contrôlées, refus motivé, paiement atomique : mouvement out + écriture comptable) ; contrôleurs/routes/permissions `depenses.*` ; notification validateur (placeholder event, branchée au centre S9) ; upload justificatif via `/uploads` ; tests workflow complet + refus de transition invalide. □ Transition `draft` $\rightarrow$ `approved` directe impossible.

□ Après `pay` : solde du compte baissé du montant exact, transaction comptable présente.

11.2 Tâche T2.2 — Règles de commissions versionnées (BE, BS, 4 j)

Tables M3 ; extension `CommissionService` : à l'encaissement, évaluer les règles actives matchant (périmètre, service, trigger) et créer les `commission_entries` avec `rule_snapshot` ; fallback existant

préservé ; endpoints CRUD (update = nouvelle version) + entries + validate/pay ; tests multi-règles et paliers. ☐ Modifier une règle ne change PAS les entrées passées (snapshot).
☐ Deux règles matchant une même vente $\Rightarrow$ deux entrées.

11.3 Tâche T2.3 — UI Dépenses (FE, LM, 5 j)

Pages : liste avec onglets par statut (badges colorés), formulaire demande (catégorie Select depuis `accounting-categories` type expense, montant, date, compte visé, justificatif upload, note), file de validation (actions Valider/Rejeter avec raison), vue détail avec historique des statuts. Menu FINANCE/Dépenses, permissions alignées.

11.4 Tâche T2.4 — UI Commissions (FE, LM, 4 j)

Page COMMISSIONS : onglet Règles (liste + modale création/édition : bénéficiaire, périmètres, formule percent/fixed/tiers avec éditeur de lignes de palier, dates, trigger ; avertissement « édition = nouvelle version » + historique versions visible), onglet Commissions (entries paginées, filtres période/statut, actions Valider/Payer, aperçu du snapshot de règle appliquée).

11.5 Tâche QA-C2/C3 — QA binôme (1 j)

Parcours : règle 5 % Academy Douala $\rightarrow$ vente + encaissement $\rightarrow$ entry calculée $\rightarrow$ validation $\rightarrow$ paiement.

12 SPRINT 4 — CRM complet

Référence : modèle M4. Réutilise `prospects.api.ts` (déjà prêt).

12.1 Tâche T4.1 — Opportunités & pipeline (BE, LM, 3 j)

Table opportunities + CRUD + PATCH stage (transitions libres tracées) ; conversion gagné $\Rightarrow$ possibilité créer client/facture liée (réutiliser convert prospect) ; scopes ; tests.

12.2 Tâche T4.2 — Activités & relances (BE, LM, 3 j)

Table activities polymorphe + CRUD + POST `/activities/{id}/complete` ; job quotidien de rappel (console.php 07 :00) créant les notifications (service placeholder si M9 pas encore là : log + table prête).

12.3 Tâche T4.3 — Entreprises & dédoublonnage (BE, LM, 2 j)

Table companies + CRUD + liaison prospect.company_id ; recherche clients/prospects normalisée téléphone (suffixe 8 chiffres) ; tests doublons.

12.4 Tâche T4.4 — Kanban prospects (FE, BS, 4 j)

Page PROSPECTS (manquante !) : kanban 7 colonnes (stages), drag & drop HTML5 natif (aucune lib), carte = prospect/opportunité (nom, montant attendu, âge), filtres (commercial, agence, département, recherche), compteur par colonne ; menu CRM global + pays + agence.

12.5 Tâche T4.5 — Fiche prospect complète (FE, BS, 3 j)

Page détail : blocs Identité / Acquisition (source) / Intérêt (types cochables) / Responsable / Pipeline (stage + montant) / Activités (timeline + formulaire rapide appel/RDV/note/reliance avec échéance) ; bouton Convertir en client (existante) ; i18n.

12.6 Tâche T4.6 — Timeline client (FE, BS, 2 j)

Onglet timeline sur ClientDetailPage : activités + factures + paiements + opportunités fusionnés chronologiquement (/crm/timeline).

12.7 Tâche QA-C4 — QA binôme (1 j)

Parcours CDC prospect → qualification → proposition → gagné → client.

13 SPRINT 5 — Academy compléments & Contrats Agency

Références : modèles M5, M6.

13.1 Tâche T5.1 — Présences par session (BE, BS, 2 j)

Table attendances + PUT /training-sessions/{session}/attendances (bulk upsert, valide enrollment appartient à la session) ; garde compat booléen enrollments.attendance synchronisé (présent si dernière présence = present/late) ; tests.

13.2 Tâche T5.2 — Certificats (BE, BS, 2 j)

Table certificates ; émission si enrollment completed ; numéro CERT- ; révocation motivée ; route PDF imprimable simple (vue Blade print) ; permissions certificats.* ; tests.

13.3 Tâche T6.1 — Contrats & renouvellements (BE, BS, 4 j)

Tables M6 ; ContractService : statuts calculés par job 06 :30 (due_soon à J-X depuis setting renew_alert_days, expired après end_date) ; renew (nouveau contrat chaîné parent_contract_id, renewal_count++) ; terminate ; is_pass_through sur catégories comptables + exclusion du CA dans StatsController (CA net du pass-through) ; tests renouvellement et exclusion CA.

13.4 Tâche T5.3 — UI Academy compléments (FE, LM, 5 j)

Feuille de présence session (tableau apprenants + 4 statuts radio, sauvegarde bulk, indicateur taux présence) ; onglet Certificats fiche apprenant (émis/revoqué, imprimable) ; bloc Finance fiche apprenant : échéancier = liste des paiements de la facture liée + solde calculé (déjà partiel, fiabiliser via enrollments.invoice_id).

13.5 Tâche T6.2 — UI Contrats & Renouvellements (FE, LM, 4 j)

Menu département type agency : Contrats (liste statuts colorés, modale création client+pack+services+prix+c détail avec services et historique renouvellements chaînés) ; Renouvellements (file due_soon/expired avec actions Renouveler/Résilier, badges J-x).

13.6 Tâche QA-C5/C6 — QA binôme (1 j)

Parcours CDC : contrat → alerte J-7 simulée (job manuel tinkér) → renouvellement → chaîne visible.

14 SPRINTS 6–7 — Store / Matootoo

Référence : modèles M7. Tout le Store vit sous le département type `store`.

14.1 Tâche T7.1 — Fournisseurs & achats (BE, LM, 4 j)

Tables `suppliers/purchases/purchase_lines`; `receive` (qty par ligne, statut partial/received, mouvements in via `StockService`, MAJ `purchase_price` produit si fourni); `pay` fournisseur (sortie trésorerie, `payment_status`); `permissions`; tests réception partielle.

14.2 Tâche T7.2 — Stocks (BE, LM, 4 j)

Tables `stock_levels/stock_movements` + `StockService` (transaction : mouvement → recalcul niveau → alerte seuil); endpoints `levels/movements/movement` manuel; transfert inter-agences (paire `transfer_out/transfer_in` même `transfer_ref`, 2 appels service atomiques); tests invariants (somme mouvements = niveau).

14.3 Tâche T7.3 — UI Catalogue & Stocks (FE, BS, 5 j)

Sous dept store : Catalogue (produits existants API + catégories; SKU, prix achat/vente, gestion stock on/off, dispo agence), Stocks (niveaux avec badge sous-seuil rouge, journal mouvements filtrable, modales ajustement/perte/transfert guidé 3 étapes).

14.4 Tâche T7.4 — UI Fournisseurs & Achats (FE, BS, 4 j)

Fournisseurs CRUD; Achats : liste statuts, création multi-lignes produits (Autocomplete produits), réception par ligne (qty reçue), paiement fournisseur, impression bon.

14.5 Tâche T7.5 — Commandes, livraisons, retours (BE, BS, 4 j)

`order_lines.product_id`; commande store confirmée → réservation simple v1; deliveries CRUD/statuts; returns workflow complet (accept → restock via `StockService`; refund → sortie trésorerie); tests impacts stock/trésorerie.

14.6 Tâche T7.6 — Inventaires (BE, BS, 3 j)

Tables + open (gèle expected), counting (saisie counted), close (écarts → mouvements ajustement valorisés); `permissions`; tests clôture avec écarts.

14.7 Tâche T7.7 — Rapports store (BE, BS, 1 j)

GET `/reports/store` : rotation (qté vendue/stock moyen période), marge ((PV-PA)/PV pondérée), top produits, valeur du stock.

14.8 Tâche T7.8 — Ventes Store POS (FE, LM, 3 j)

Adapter QuickSale : onglets Services/Produits ; panier produits ; validation $\Rightarrow$ order + invoice + paiements + mouvements **sale** (backend orchestré dans OrderController confirm/invoice) ; stock temps réel dans le panier (badge rupture).

14.9 Tâche T7.9 — UI Commandes/Livraisons/Retours (FE, LM, 4 j)

Trois pages menu store : commandes (statuts, génération livraison), livraisons (kanban statuts simple + scan réf), retours (demande, traitement accept/reject, remboursement avec compte).

14.10 Tâche T7.10 — UI Inventaires & rapports (FE, LM, 2 j)

Inventaires (sessions, saisie comptage mobile-friendly, écran écarts valorisés avant clôture) ; Rapports store (cartes rotation/marge/valeur stock + top produits).

14.11 Tâche QA-C7 — QA binôme (2+1 j)

Parcours CDC complet : achat 50 unités $\rightarrow$ vente 3 $\rightarrow$ retour 1 $\rightarrow$ inventaire $\rightarrow$ ajustement $\rightarrow$ rapports cohérents.

15 SPRINTS 8–9 — Studio & Notifications

Références : modèles M8, M9. Sémantique : projet = production concrète (ex. film publicitaire) ; devis = calcul du montant ; briefs = réunions avec notes Summernote.

15.1 Tâche T8.1 — Devis (BE, LM, 4 j)

Tables quotes/quote_lines ; transitions send/accept/refuse datées ; expiration (valid_until, job) ; convert $\rightarrow$ project planning (copie lignes en description) ; permissions **devis.*** ; tests conversion.

15.2 Tâche T8.2 — Projets, briefs, planning (BE, LM, 4 j)

Tables projects/project_briefs/project_tasks ; réordonnancement tasks (position) ; statuts contrôlés ; sanitisation HTML briefs (**purifier** ou **strip_tags** liste blanche) ; permissions ; tests.

15.3 Tâche T8.3 — UI Devis & Projets (FE, BS, 5 j)

Devis : liste pipeline coloré, éditeur lignes dynamique (label/prix/qté/remise/TVA, total live), aperçu imprimable, actions Send/Accept/Refuse/Convertir. Projets : cartes statuts (pipeline), détail avec onglets.

15.4 Tâche T8.4 — UI Briefs & Planning (FE, BS, 4 j)

Briefs : liste réunions du projet + éditeur **Summernote** (installer **react-summernote** + jQuery peer, décision validée car exigence métier ; fallback textarea HTML si build KO) ; participants, date ; rendu HTML consultable. Planning : tâches en vue calendrier mois simple (grille CSS native) + liste drag & drop statuts.

15.5 Tâche T8.5 — Production, révisions, livraisons, clôture (BE, BS, 4 j)

Endpoints revisions (round auto-incrémenté par projet), deliverables (upload, version++), close (exige facture solde payée ou solde zéro ; passe delivered→closed) ; acompte : à la conversion, générer facture acompte (50 % paramétrable settings) liée `invoice_deposit_id` ; tests cycle complet CDC.

15.6 Tâche T8.6 — UI Production/Révisions/Livraisons (FE, LM, 5 j)

Onglets projet : Production (tâches en cours, avancement %), Révisions (cycles demandeur/-résolution), Livraisons (upload livrables versionnés, bouton livrer → statut delivered), Paiements (acompte/solde reliés factures, bouton encaisser réutilisant composant S2).

15.7 Tâche T9.1 — Centre de notifications (BE, BS, 4 j)

Table + `NotificationService` ; brancher TOUS les déclencheurs M9 (dépense soumise, stock bas, créances échues, renouvellements, rappels activités, objectif) ; endpoints liste/read-all/patch ; job 07 :00 consolidé ; rate-limit anti-spam (unique user+type+entité+jour).

15.8 Tâche T9.2 — Cloche & centre (FE, LM, 4 j)

Cloche ContextBar (polling 60 s, badge count) ; panneau latéral notifications (non-lues en gras, clic ⇒ navigation entité liée) ; page centre avec tout-lu et filtres ; i18n.

15.9 Tâche QA-C8/C9 — QA binôme (1+1 j)

Parcours CDC Studio complet prospect→clôture + scénario notifications multi-déclencheurs.

16 SPRINT 10 — Command Center, rapports & migration

Référence : modèle M10. Décision requise : définitions KPI finales (sinon provisoires documentées).

16.1 Tâche T10.1 — API KPI consolidée (BE, LM, 4 j)

`GET /command-center` : CA (net pass-through), encaissements, créances, dépenses, trésorerie par compte, performance (par pays/agence/dépt/commercial), alertes (top 10) ; chaque valeur = série comparable période précédente ; `GET /kpi/drill-down?dimension=...` descente Groupe→Pays→Agence→D (payload borné : 100 lignes max par niveau) ; cache 60 s.

16.2 Tâche T10.2 — Exports & importeur Sheets (BE, LM, 4 j)

Etendre `ExportController` (command-center, treasury, expenses, commissions, contracts, store-report, studio-report) Excel/PDF/CSV ; commande `pekegno:import-sheets {file} {entity} --dry-run-force` avec mappings JSON versionnés, normalisation (dates, téléphones, montants), dédoublonnage par téléphone, `import_logs` ; entités supportées v1 : clients, prospects, produits, formations.

16.3 Tâche T10.3 — Command Center (FE, BS, 5 j)

Redesign `PEKEGNOGroupDashboard` : cartes KPI cliquables (delta % vs période précédente, flèche tendance), drill-down au clic (modal/tableau niveau suivant jusqu'à la transaction avec lien facture),

section Alertes actionable, tout piloté par ContextBar (pays/agence/dépt/période); graphiques : réutiliser le pattern CSS bars existant ou ajouter recharts (décision rapide à trancher, défaut : CSS).

16.4 Tâche T10.4 — Rapports transverses (FE, BS, 3 j)

Page RAPPORTS : filtres communs (période PeriodPicker, pays, agence, département, commercial) persistés dans l'URL; onglets Direction/Commercial/Finance/Academy/Agency/Store/Studio consommant les endpoints existants + nouveaux; boutons export.

16.5 Tâche T10.5 — Recette générale (QA, binôme, 2 j)

Exécuter les 7 parcours critiques du CDC de bout en bout sur données de démo; corriger les bloquants trouvés; checklist DoD §8 complète; rapport final.

17 Annexe — Interdits et pièges connus

- Ne PAS réutiliser `AdminDashboardPage` / `DashboardPlaceholderPage` (orphelins legacy) : partir du composant existant group dashboard.
- `CatalogSeeder` existe mais n'est pas appelé par `DatabaseSeeder` : l'appeler si la tâche a besoin de catalogue de démo.
- Le frontend n'a AUCUN framework de test : ne pas en introduire dans une tâche métier (décision séparée).
- Pas de spatie/laravel-permission : RBAC maison (`roles`, `permissions`, `role_permission`, `users.role_id`).
- Portails Sanctum distincts staff/client (`portal:staff`) : ne jamais exposer de routes business côté `portal:client` sans décision.
- `agencies.type` déprécié dès S1; ne plus s'y fier dans le moindre nouveau code.
- Les montants s'affichent en FCFA via `formatCurrency`; ne jamais formater à la main.
- Toute route nouvelle doit apparaître dans `route:list` ET Swagger; toute clé i18n doit exister dans `fr.ts` ET `en.ts`.